2024A7PS0002G
AI Lab5: BN

1. In bn_firstorder.py

2. In bn_secondorder.py

3. Conditional Probability Tables
   Firstorder:

```
P(next word | <START>, <START>)
the: 1.000

P(next word | <START>, the)
cat: 0.500
dog: 0.500

P(next word | the, cat)
sat: 0.667
ran: 0.333

P(next word | the, dog)
sat: 0.667
ran: 0.333

P(next word | cat, sat)
on: 1.000

P(next word | dog, sat)
on: 1.000

P(next word | dog, ran)
to: 1.000

P(next word | sat, on)
the: 1.000

P(next word | ran, to)
the: 1.000
```

   Secondorder:

```
P(next word | <START>, <START>)
the: 1.000

P(next word | <START>, the)
cat: 0.500
dog: 0.500

P(next word | the, cat)
sat: 0.667
ran: 0.333

P(next word | the, dog)
sat: 0.667
ran: 0.333

P(next word | cat, sat)
on: 1.000

P(next word | dog, sat)
on: 1.000

P(next word | dog, ran)
to: 1.000

P(next word | sat, on)
the: 1.000

P(next word | ran, to)
the: 1.000
```

4. Examples of generated text
   Firstorder:

```
20 Sampling-generated sentences:
1. the dog sat on the park
2. the rug
3. the park
4. the park
5. the dog sat on the mat
6. the park
7. the rug
8. the rug
9. the cat ran to the park
10. the park
11. the rug
12. the park
13. the rug
14. the cat sat on the cat ran to the park
15. the dog sat on the cat ran to the park
16. the mat
17. the mat
18. the dog ran to the dog sat on the dog sat on the cat sat on the park
19. the dog ran to the dog sat on the cat sat on the mat
20. the cat sat on the park
```

```
5 Greedy-generated sentences:
1. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
2. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
3. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
4. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
5. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat

5 Sampling-generated sentences:
1. the cat sat on the park
2. the rug
3. the park
4. the cat sat on the park
5. the rug
```

Secondorder:

```
20 Sampling-generated sentences:
1. the cat sat on the mat
2. the cat ran to the park
3. the dog ran to the park
4. the dog ran to the park
5. the dog sat on the rug
6. the cat sat on the mat
7. the cat ran to the park
8. the cat ran to the park
9. the dog sat on the rug
10. the cat sat on the mat
11. the cat sat on the mat
12. the cat sat on the rug
13. the cat sat on the rug
14. the cat sat on the rug
15. the dog ran to the park
16. the dog sat on the mat
17. the dog ran to the park
18. the dog ran to the park
19. the cat sat on the mat
20. the cat ran to the park
```

```
5 Greedy-generated sentences:
1. the cat sat on the mat
2. the cat sat on the mat
3. the cat sat on the mat
4. the cat sat on the mat
5. the cat sat on the mat

5 Sampling-generated sentences:
1. the dog sat on the mat
2. the dog sat on the rug
3. the dog ran to the park
4. the dog ran to the park
5. the cat ran to the park
```

5. Probability Normalization tests
   Firstorder:

```
Probability normalisation test:
<START>: 1.000
the: 1.000
cat: 1.000
sat: 1.000
on: 1.000
mat: 1.000
rug: 1.000
dog: 1.000
ran: 1.000
to: 1.000
park: 1.000
```

   Secondorder:

```
Probability normalisation test:
('<START>', '<START>'): 1.000
('<START>', 'the'): 1.000
('the', 'cat'): 1.000
('cat', 'sat'): 1.000
('sat', 'on'): 1.000
('on', 'the'): 1.000
('the', 'mat'): 1.000
('the', 'rug'): 1.000
('the', 'dog'): 1.000
('dog', 'sat'): 1.000
('dog', 'ran'): 1.000
('ran', 'to'): 1.000
('to', 'the'): 1.000
('the', 'park'): 1.000
('cat', 'ran'): 1.000
```

6.1) Decomposition is useful for generating text as it allows us to generate one word at a time based on previous words. It helps break generation into smaller, simpler problems.

6.2) The first-order Markov assumption is that the next word depends only on the immediately previous word.

6.3)

```
P(next word | the)
cat: 0.250
mat: 0.167
rug: 0.167
dog: 0.250
park: 0.167

P(next word | cat)
sat: 0.667
ran: 0.333

P(next word | dog)
sat: 0.667
ran: 0.333

P(next word | sat)
on: 1.000

P(next word | ran)
to: 1.000
```

Zero-probability:

For current word = the:
P(sat | the) = 0
P(ran | the) = 0
P(on | the) = 0
P(to | the) = 0

For current word = cat:
P(cat | cat) = 0
P(dog | cat) = 0
P(mat | cat) = 0
P(rug | cat) = 0
P(park | cat) = 0
P(on | cat) = 0
P(to | cat) = 0

For current word = dog:
P(cat | dog) = 0
P(dog | dog) = 0
P(mat | dog) = 0
P(rug | dog) = 0
P(park | dog) = 0
P(on | dog) = 0
P(to | dog) = 0

For current word = sat:
P(cat | sat) = 0
P(dog | sat) = 0
P(mat | sat) = 0
P(rug | sat) = 0
P(park | sat) = 0
P(ran | sat) = 0
P(to | sat) = 0

For current word = ran:
P(cat | ran) = 0
P(dog | ran) = 0
P(mat | ran) = 0
P(rug | ran) = 0
P(park | ran) = 0
P(sat | ran) = 0
P(on | ran) = 0

6.4) The transition counts are stored by:
transition_counts = defaultdict(Counter)
transition_counts[current_word][next_word] += 1

6.5) It is computed when the program converts transition counts into probabilities.

6.6) Greedy: the most probable word as per the weights
Sampling: from the probability distribution

6.7) If the program encounters a word for which no transition has been observed, that word will not exist in probabilities. The program will stop generation.

6.8) The totals must add up to 1, so this shows that the conditional probability distribution is not properly normalized.

6.9) Probability model reflects patterns in its training data, while human linguistic expectations use more complicated rules and context that a model can only mimic.

6.10) Sampling produces more variation due to induced randomness. As observed in the results, greedy is deterministic on a particular training set.

6.11) First order depends only on previous 1 word, second order depends on previous 2 words.

The second-order conditional probability table is larger as it stores combinations of previous 2 words.
The second order has a richer context.
The second order requires more training data.

6.12) The size of conditional probability table shows that a model has more info about what came before, but the model will need more training data to set its parameters.

6.13) Approach B is preferable because it clearly specifies what probabilistic model should be implemented and how it should behave, instead of leaving the LLM to decide what “a language model” means.
The LLm has more info about the intended behaviour.
Testing probabilistic invariants means checking whether the sum of all the probabilities of next word add up to 1, which serves as a check.
The model refers to the “weights” and probabilities while the python code is how it is implemented.

6.14) Thinking of the language model as a Bayesian network shows that generation can be achieved by mapping a probability function and building on previous words.

Understanding the effect of increasing context:
Comparing first-order and second order networks shows what happens when more context is used. A second-order model can make more specific and often better predictions because it uses two previous words. However, it also requires a larger conditional probability table and more data to estimate its probabilities reliably.

Testing whether the implementation matches the specification:
Thinking probabilistically gives us properties that the program should satisfy. eg: for every context, the probabilities of all possible next words should sum to 1.

A principled method for generation:
Text generation follows directly from the model. The program samples potential next words based on a probability distribution given the previous words.

7. I used the LLM to help generate the Python code for both the first-order and second-order models after specifying the required behaviour. I verified and tested the code.
An example of a piece of LLM-generated code that I corrected was the Greedy generation, being deterministic, was programmed to stop only when {END} was the most probable token. The weights set using training examples on the first-order model caused {END} to never be chosen on this small vocabulary set, which led to never-ending generation. I fixed this by implementing a hard cap on the number of words generated.